

فصل نهایی: مهندسی ایچنت

LangChain, CrewAI & Observability with LangFuse

دوره جامع توسعه هوشمند بر پایه Google Gemini

اکوسیستم فریم‌ورک‌های ایجنتیک (2025)

	 LangGraph	 CrewAI	 Autogen
	LangGraph	CrewAI	Autogen
Developer Experience	High flexibility, steep learning curve, limited pre-built components	Easy to use, good documentation, fast setup	Moderate ease of use, well-structured documentation
Agent Capabilities	Highly customizable, strong task execution and tool usage, supports complex agent hierarchies	Stable task planning and execution, good collaboration through shared memory and messaging	Strong multi-agent collaboration, excellent task planning and execution
Enterprise Features	Requires additional development for enterprise features	Limited, with advanced security features only in paid version	Good enterprise support, including security and logging features
Strengths	Flexible and powerful, supports custom behaviors at any node	Great team-based agent collaboration, easy configuration by non-technical users	High extensibility, supports complex workflows, strong tooling support
Weaknesses	Hard to learn, requires building many components from scratch	Limited extensibility, poor logging capabilities	Requires familiarity with the ecosystem, initial setup complexity
Best Use Cases	Complex agent systems with custom interaction and tool usage patterns	Rapid development of simple agent applications or apps configured by non-technical users	Advanced multi-agent systems, enterprise-grade applications with robust security and logging

انتخاب ابزار صحیح، تفاوت بین یک **تست ساده (PoC)** و یک **سیستم عملیاتی (Production)** است.

- **LangGraph**: متمرکز بر گراف‌های حالت‌دار (Stateful) و کنترل دقیق.
- **CrewAI**: متمرکز بر نقش‌آفرینی (Role-playing) و فرآیندهای تیمی.
- **AutoGen**: متمرکز بر مکالمات چندجانبه و الگوهای منعطف.

کدام فریم‌ورک برای پروژه شما مناسب است؟

ویژگی	LangGraph	CrewAI	AutoGen
مدل ذهنی	Graph / State Machine	Role-based Crew	Conversational Agents
مدیریت وضعیت (State)	بسیار دقیق و ماندگار	اشتراک‌گذاری در Context	تاریخچه پیام‌ها
مناسب برای...	گردش‌کارهای پیچیده مهندسی	تولید محتوا و تحقیق	توسعه کد و حل مسئله
کنترل (Control)	حداکثری (Low-level)	متوسط (High-level)	داینامیک

تمرکز بر LangGraph: قدرت در گراف‌ها

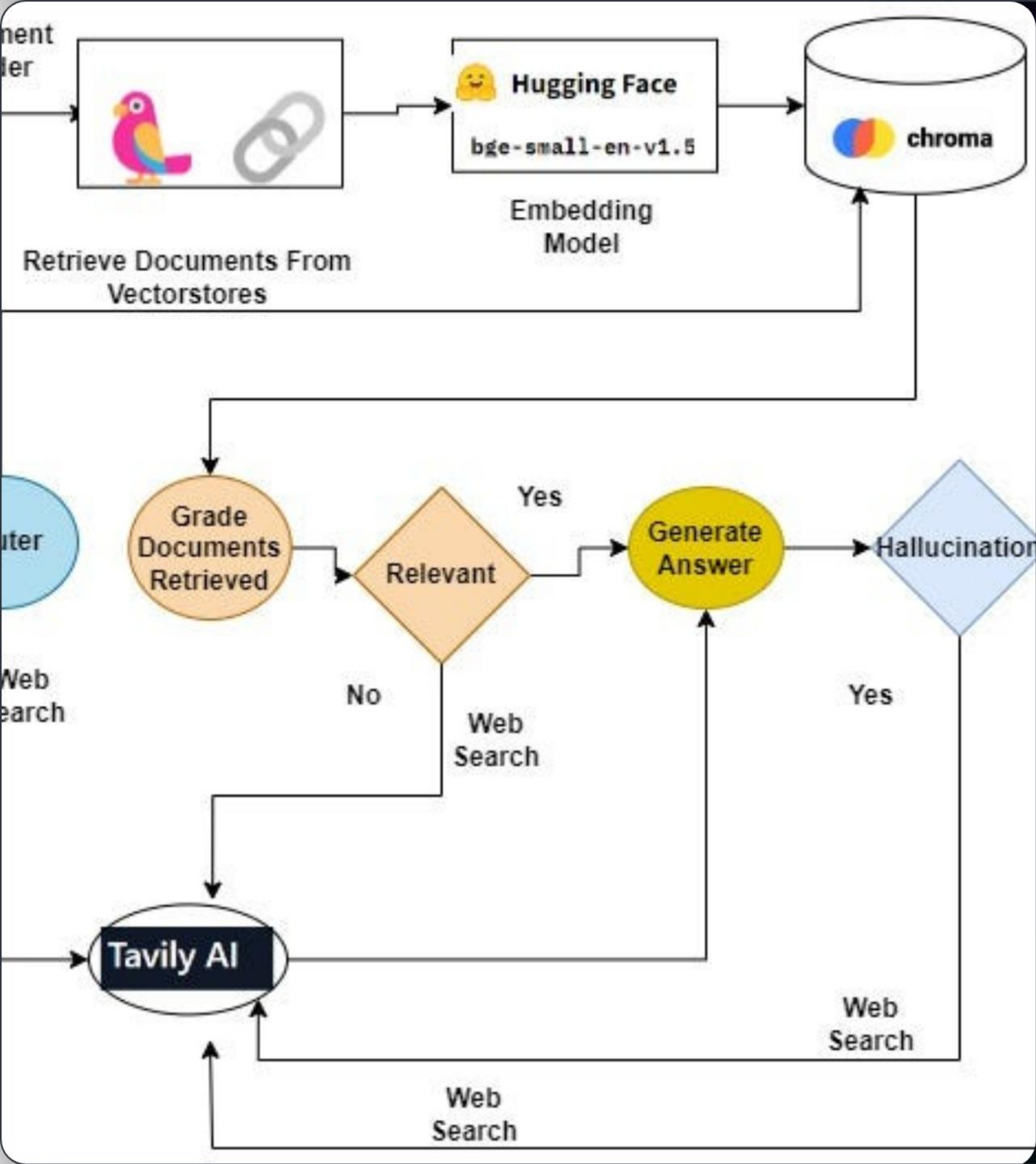
چرا LangGraph لیدر است؟

برخلاف زنجیره‌های خطی ساده، LangGraph اجازه می‌دهد ****حلقه‌ها (Loops)**** را پیاده‌سازی کنید. این یعنی ایجنت می‌تواند نتیجه کار خود را نقد کند و دوباره تلاش کند.



کلید واژه: **Cyclic Graphs**

- **Nodes**: توابعی که منطق یا ابزارها را اجرا می‌کنند.
- **Edges**: مسیرهای انتقال بین گره‌ها.
- **State**: آبجکت مرکزی که داده‌ها در آن ذخیره می‌شوند.



مفهوم State و Reducers

در LangGraph، وضعیت سیستم یک ****تایپ دیکشنری**** است که توسط تمام نودها خوانده و نوشته می‌شود.

```
class AgentState(TypedDict):
    messages: Annotated[list, add_messages]
    is_finished: bool
```

تابع **add_messages** یک Reducer است که پیام‌های جدید را به لیست قبلی اضافه می‌کند بدون اینکه تاریخچه را پاک کند.

ساختارهای موازی: Fan-out / Fan-in

بهینه‌سازی سرعت اجرا

زمانی که ایجنت باید چند کار مستقل را همزمان انجام دهد (مثلاً جستجو در ۳ منبع مختلف)، از الگوی Fan-out استفاده می‌کنیم.

- کاهش چشمگیر Latency سیستم.
- جمع‌آوری نتایج در یک نود "ارکستراتور" (Fan-in).
- استفاده از **Parallel Map** برای اجرای داینامیک.



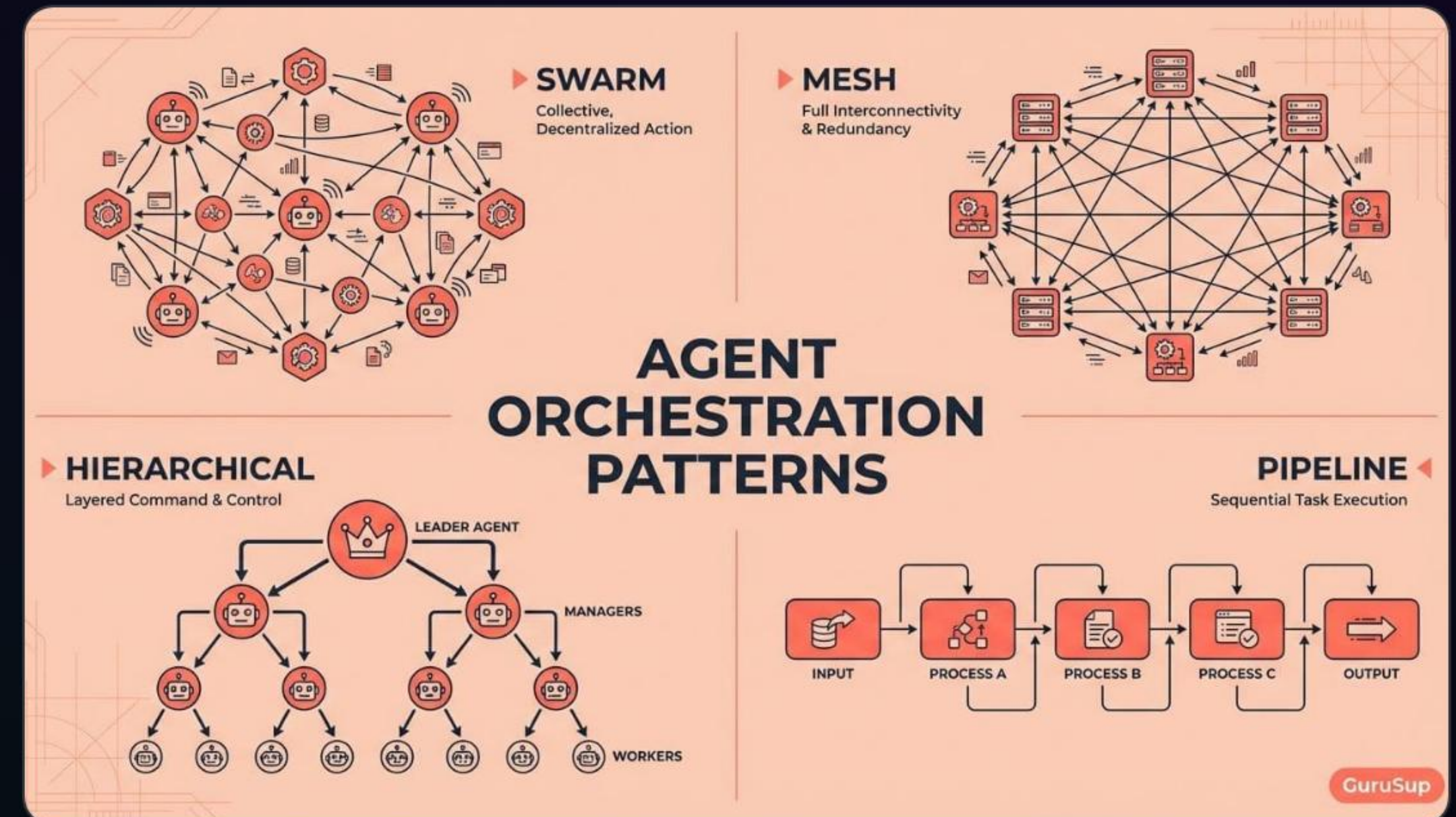
در LangGraph، با تعریف چند Edge خروجی از یک نود، اجرای موازی به صورت بومی مدیریت می‌شود.

الگوهای ارکستراسیون: Supervisor

مدل متمرکز (Centralized)

یک ایجنت "ناظر" تصمیم می‌گیرد که تسک را به کدام ایجنت "کارگر" (Worker) بسپارد.

- ناظر ورودی را تحلیل می‌کند.
- به ترتیب یا موازی به ایجنت‌های تخصصی (کدنویس، محقق، دی‌زاینر) تسک می‌دهد.
- خروجی نهایی را جمع‌بندی و بازبینی می‌کند.



ارکستراسیون سلسله مراتبی (Hierarchical)

مقیاس پذیری

مناسب برای پروژه‌های بسیار بزرگ که نیاز به ۲۰+ ایجنت در دامنه‌های مختلف دارند.

کاهش Context

هر سطح فقط اطلاعات مربوط به وظیفه خودش را می‌بیند، که باعث کاهش هزینه و خطا می‌شود.

تیم‌های تودرتو

ناظران می‌توانند خودشان زیر-گروه‌هایی از ایجنت‌ها را مدیریت کنند.

معرفی CrewAI: مدیریت تیم‌های ایجنتی



CrewAI بر پایه ****نقش‌ها (Roles)**** و ****تسک‌ها (Tasks)**** بنا شده است. پیاده‌سازی آن بسیار سریع‌تر از LangGraph است اما کنترل کمتری روی جزئیات اجرای هر استپ دارید.

- **Manager LLM**: مدیریت خودکار جریان کار.
- **Processes**: تعریف شیوه همکاری (Sequential, Hierarchical, Consensual).
- **Tools**: قابلیت استفاده از ابزارهای LangChain.

چالش بزرگ: ایجنت من چه می‌کند؟



در سیستم‌های ایجنتیک، "دیباگ کردن" سخت‌ترین کار است.

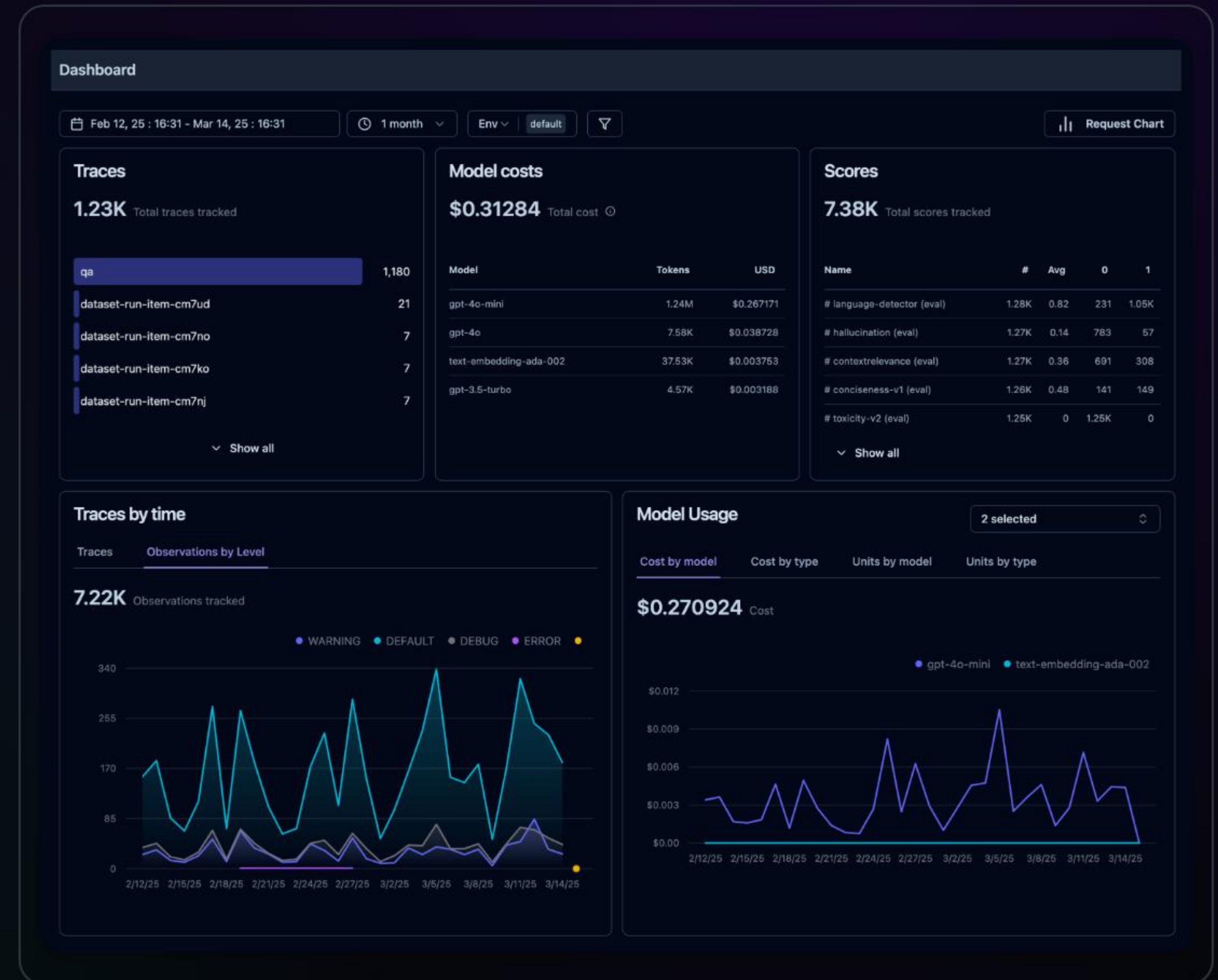
چرا پاسخ غلط بود؟ کدام ایجنت اشتباه کرد؟ هزینه این اجرا چقدر شد؟

نظارت (Observability) با LangFuse

چرا LangFuse؟

LangFuse یک پلتفرم Open-source برای تریسینگ (Tracing) و ارزیابی سیستم‌های هوشمند است.

- **Tracing**: مشاهده درخت اجرای کامل (Chain of Thought).
- **Latencies**: مانیتورینگ سرعت پاسخ‌دهی هر نود.
- **Costs**: محاسبه دقیق هزینه توکن‌ها بر اساس مدل.



مدیریت پرامپت در LangFuse



Prompt Fetching

ایجنت شما همیشه آخرین ورژن تایید شده را از API دریافت می‌کند.



A/B Testing

تست چند ورژن پرامپت به صورت همزمان برای یافتن بهترین عملکرد.



نسخه‌بندی (Versioning)

بدون تغییر کد، پرامپت‌ها را در AI آپدیت کنید و به ورژن قبل برگردید.

ردیابی (Tracing) در سیستم‌های چندایجنت

Integration Snippet

```
(...)langfuse_handler = CallbackHandler  
graph.invoke(input, config={"callbacks": [langfuse_handler]})
```

زمانی که از LangGraph استفاده می‌کنید، LangFuse تمام گره‌ها را به صورت شاخه‌دار نشان می‌دهد.

- مشاهده ورودی/خروجی هر Tool Call.
- شناسایی حلقه‌های بی‌نهایت (Infinite Loops).
- امکان امتیازدهی (Scoring) به پاسخ‌ها توسط کاربر یا ایجنت دیگر.

تجسم ساختارهای موازی در LangFuse



تریس‌های موازی به شما نشان می‌دهند که آیا تمام شاخه‌ها با موفقیت به "Arbiter" یا نود تجمع رسیده‌اند یا خیر.

ارزیابی عملکرد (Evaluation)

بهبود مداوم

استفاده از سیستم‌های ****LLM-as-a-Judge**** در LangFuse برای
ارزیابی خودکار کیفیت پاسخ‌ها بر اساس پارامترهایی مثل:

- صداقت (Faithfulness)
- ارتباط با سوال (Relevance)
- رعایت قوانین امنیتی (Guardrails)

98%

خلاصه ابزارهای مهندسی ایجنت

Observability

Tracing & Debugging

Token & Cost Analytics

Management

LangFuse Prompt UI

Version Control

Core Logic

LangGraph (Stateful)

CrewAI (Teamwork)

معماری پروژه نهایی دوره

- مدل: Google Gemini 1.5 Pro
- ارکستراتور: LangGraph (ساختار گراف Cyclic)
- حافظه: RAG با دیتابیس برداری (Vector DB)
- نظارت: اتصال کامل به LangFuse برای مانیتورینگ عملیاتی
- رابط کاربری: Antigravity UI Dashboard

پایان دوره - آغاز توسعه

اکنون شما تمام ابزارهای لازم برای ساخت ایجنت‌های کلاس جهانی را دارید.



سوالی دارید؟ پروژه نهایی را چگونه می‌خواهید توسعه دهید؟

Image Sources

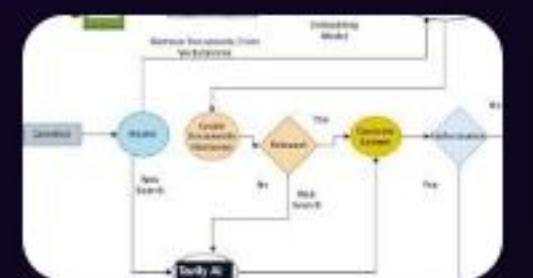
https://miro.medium.com/v2/resize:fit:1400/1*wA7TxnVwT2DKohkuwgXYmg.png

Source: aaronyuqi.medium.com



https://miro.medium.com/1*2M6Zd6w0L78fXQqKBWwFtg.jpeg

Source: medium.com



https://gurusup.com/_next/image?url=https%3A%2F%2Fres.cloudinary.com%2Fdhqzl4gpk%2Fimage%2Fupload%2Fv1773141256%2Ffeatured_agent_orchestration_patterns_5547b329ca.jpg&w=3840&q=75

Source: gurusup.com



https://images.prismic.io/element451/aB4OVydWJ-7kR1fT_May6%2C2025%2C11_25_31AM.png?auto=format&ixlib=react-9.0.3&h=1024&w=1536

Source: element451.com



https://langfuse.com/images/cookbook/integration_openai-agents/dashboard-dark.png

Source: langfuse.com

